

Mastering SQL Joins

Training for Data Professionals

Level: Beginner to Intermediate

Today's Agenda

01

Why Joins Matter

Real-world data lives in multiple tables

02

INNER JOIN

Find matching records between two tables

03

LEFT & RIGHT JOIN

Include all records from one side

04

FULL OUTER JOIN

Combine everything from both tables

05

SELF JOIN & CROSS JOIN

Advanced join patterns

06

Hands-On Lab

Practice with real business scenarios

Why Joins Matter

MODULE 1

customers

cust_id	name	city
1	Priya Mehta	Mumbai
2	Raj Kumar	Delhi
3	Anita Shah	Bangalore
4	Vikram Nair	Chennai

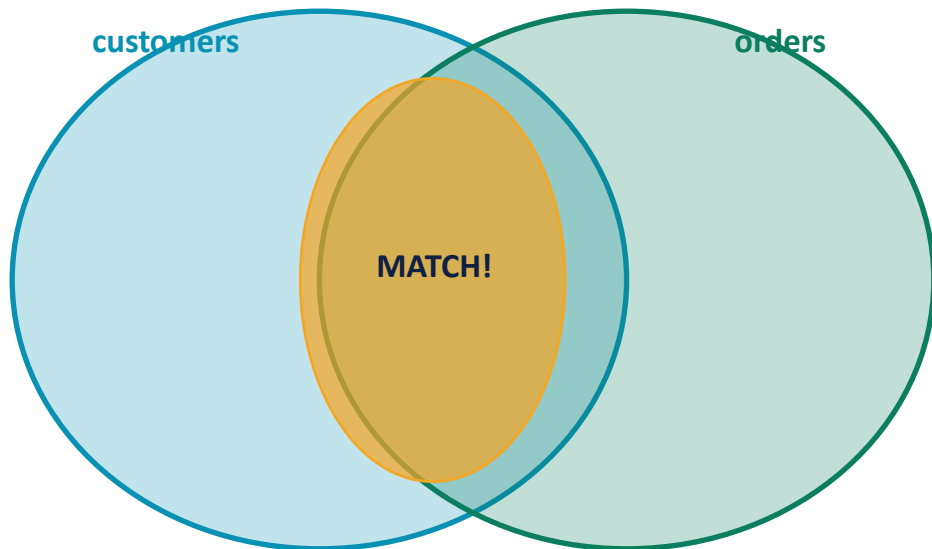
JOIN

orders

order_id	cust_id	amount
101	1	₹4,500
102	2	₹1,200
103	1	₹8,900
104	5	₹3,300

Real-world
data is
always
spread
across
multiple
tables!

INNER JOIN — Only Matching Records



KEY RULE

Returns ONLY rows where the join condition matches in **BOTH** tables.

If a row has no match, it is excluded from results.

Most commonly used join in production SQL!

```
SELECT c.name, o.order_id, o.amount
FROM customers c
INNER JOIN orders o ON c.cust_id = o.cust_id;
```

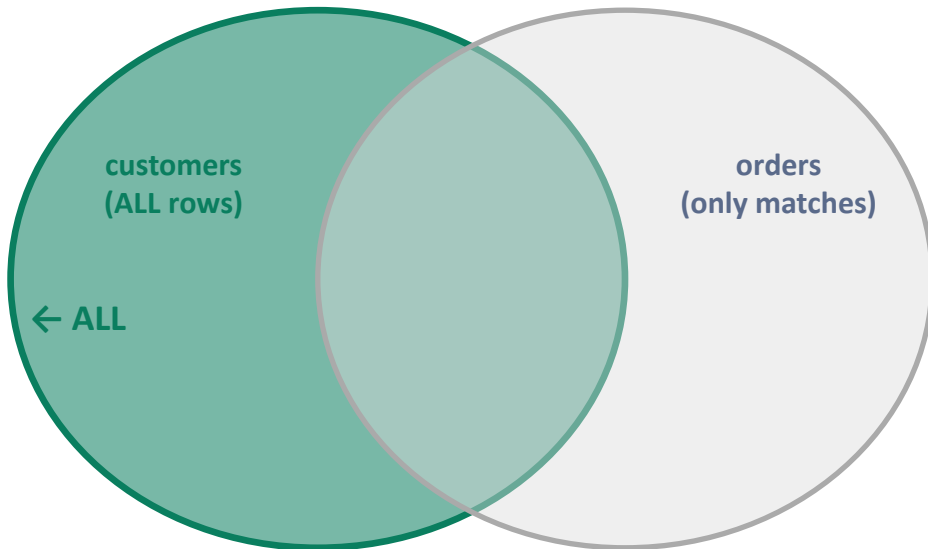
⚠ Order 104 (cust_id=5) is EXCLUDED — no matching customer!

INNER JOIN — Results in Action

name	order_id	amount	Status
Priya Mehta	101	₹4,500	✓ Matched
Raj Kumar	102	₹1,200	✓ Matched
Priya Mehta	103	₹8,900	✓ Matched
EXCLUDED from INNER JOIN result:			
Vikram Nair	—	—	✗ No orders
—	104	₹3,300	✗ No customer

3 rows
returned
out of
4 orders
& 4 customers

LEFT JOIN — Keep All Left Records

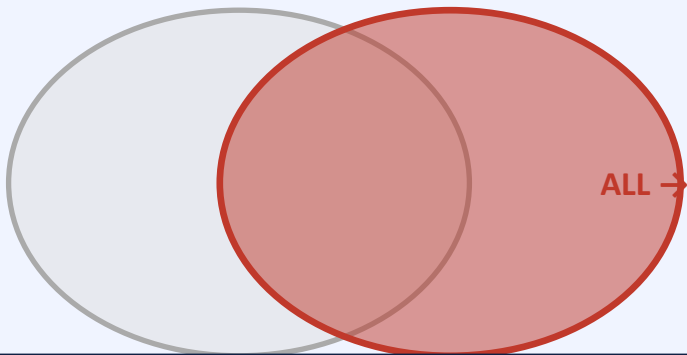


```
SELECT c.name, o.order_id, o.amount
FROM customers c
LEFT JOIN orders o ON c.cust_id = o.cust_id;
```

Result Preview	
Priya Mehta	101
Priya Mehta	103
Raj Kumar	102
Anita Shah	NULL ←
Vikram Nair	NULL ←
Anita & Vikram appear with NULL because they have no orders!	

RIGHT JOIN & FULL OUTER JOIN

RIGHT JOIN



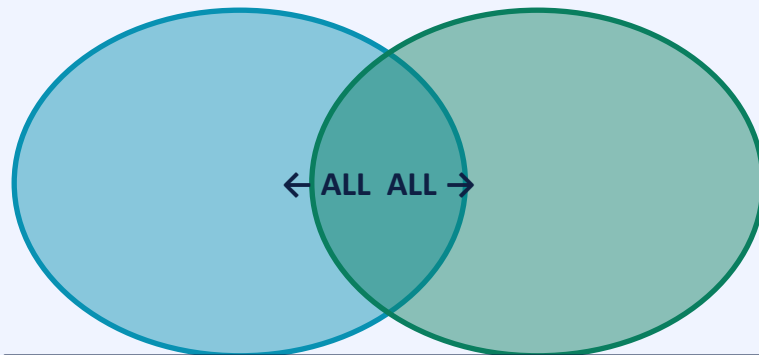
```
SELECT * FROM customers c RIGHT JOIN orders o ON  
c.cust_id = o.cust_id
```

Keeps ALL rows from the RIGHT table (orders).

Left table (customers) rows that don't match get NULL.

Use case: Find orders from unknown/deleted customers.

FULL OUTER JOIN



```
SELECT * FROM customers c FULL OUTER JOIN orders o ON  
c.cust_id = o.cust_id
```

Returns ALL rows from BOTH tables.

NULLs appear where no match exists on either side.

Use case: Data reconciliation — find unmatched records on BOTH sides.

Quick Reference: All 4 Join Types

INNER JOIN

Only matching rows

Example: Customers WITH orders

LEFT JOIN

All left + matching right

Example: All customers (+ orders if any)

RIGHT JOIN

Matching left + all right

Example: All orders (+ customer if exists)

FULL OUTER

Everything from both sides

Example: All customers AND all orders

Advanced: SELF JOIN & CROSS JOIN

MODULE 5

SELF JOIN

A table joined to ITSELF using aliases. Use when data has self-referencing relationships.

Scenario: Find each employee and their manager

```
SELECT e.name AS Employee,  
       m.name AS Manager  
FROM employees e  
LEFT JOIN employees m  
  ON e.manager_id = m.emp_id;
```

Common uses: org charts, category hierarchies, geographic parent-child data

CROSS JOIN

Every row in Table A × every row in Table B.
Returns M × N rows (Cartesian Product).

Scenario: Generate all color-size combinations

```
SELECT colors.name, sizes.label  
FROM colors  
CROSS JOIN sizes;
```

Result: 3 colors × 3 sizes = 9 rows

Red-S	Red-M	Red-L
Blue-S	Blue-M	Blue-L
Green-S	Green-M	Green-L

Hands-On Lab — E-Commerce Database

30-Minute Practice Session

01	INNER JOIN	List all customers who have placed at least one order, showing their name and total order value.  <i>Hint: JOIN on cust_id, SUM(amount)</i>
02	LEFT JOIN	Find all customers who have NEVER placed any order.  <i>Hint: LEFT JOIN + WHERE order_id IS NULL</i>
03	LEFT JOIN	Show each product and its total units sold (include products never sold).  <i>Hint: products LEFT JOIN order_items</i>
04	SELF JOIN	Display each employee alongside their manager's name from the employees table.  <i>Hint: e1.manager_id = e2.emp_id</i>

Key Takeaways

- 1 INNER JOIN → Only rows matching in both tables
- 2 LEFT JOIN → All left rows, NULL for non-matching right
- 3 RIGHT JOIN → All right rows, NULL for non-matching left
- 4 FULL OUTER → Everything from both, NULLs where no match
- 5 SELF JOIN → Table joined to itself (org charts, hierarchies)
- 6 CROSS JOIN → Cartesian product — use with caution!

Questions?